

Solving no-wait job-shop scheduling problems using a multi-start simulated annealing with bi-directional shift timetabling algorithm

Kuo-Ching Ying^a, Shih-Wei Lin^{b,c,d,*}

^a Department of Industrial Engineering and Management, National Taipei University of Technology, Taipei, Taiwan

^b Department of Information Management, Chang Gung University, Taoyuan, Taiwan

^c Department of Industrial Engineering and Management, Ming Chi University of Technology, Taipei, Taiwan

^d Department of Neurology, Linkou Chang Gung Memorial Hospital, Taoyuan, Taiwan

ARTICLE INFO

Keywords:
Scheduling
No-wait job-shop
Makespan
Metaheuristic

ABSTRACT

This paper presents a multi-start simulated annealing with bi-directional shift timetabling (MSA-BST) algorithm for solving the no-wait jobshop scheduling problem (NWJSP). To enhance the quality of the solution, a bi-directional shift timetabling procedure that allows delay timetabling and consequently enlarges the search space is embedded into the multi-start simulated annealing algorithm. The performance of the proposed MSA-BST algorithm is evaluated with respect to the makespan by comparing its computational results to those of the best-to-date meta-heuristic algorithm, the hybrid artificial bee colony (HABC) algorithm, in the literature. Experimental results that were obtained by applying the proposed algorithm to a set of benchmark problems in the literature indicate that it significantly outperforms HABC. The main contribution of this study is to provide a novel and effective alternative method for solving the complex NWJSP.

1. Introduction

This study concerns the no-wait job-shop scheduling problem (NWJSP) with the objective of minimizing makespan among all jobs, which can be denoted as the triplet $J_m|nwt|C_{max}$, where J_m denotes a machine environment that is a job-shop with m machines; nwt indicates the job-shop scheduling problem with the no-wait constraint, and C_{max} indicates that the objective is to minimize the makespan. The NWJSP is an extension of the classical job-shop scheduling problem (JSP) under the no-wait constraint, which arises when the characteristics of process technology or the absence of storage capacity requires that consecutive operations in any job are processed one after another without waiting. The $J_m|nwt|C_{max}$ problem, addressed herein, arises in several industries, including steel production, chemical and pharmaceutical manufacture, semiconductor testing, computer system development and the production of concrete wares (Zhu & Li, 2012; Sundar, Suganthan, Jin, Xiang, & Soon, 2017). Given the complexities of the $J_m|nwt|C_{max}$ problem, efficient heuristics are needed to help generate dependable solutions for practical industrial-scale applications. Although this problem has wide industrial applications, much less attention has been paid to it than has been paid to the classical JSP. This study's review of the limited studies in the literature suggests that there is room for further improving the quality of solutions to facilitate industrial applications of this emerging

scheduling problem. This is the primary motivation for this study.

To the best of our knowledge, Hall and Sriskandarajah (1996) were the first to summarize research on the NWJSP. Allahverdi (2016) comprehensively surveyed the literature on the $J_m|nwt|C_{max}$ problem. Since the $J_m|nwt|C_{max}$ problem is $\mathcal{NP}$ -hard in the strong sense (Bansal, Mahdian, & Sviridenko, 2005) when each job has at least three operations, it does not have a polynomial-time approximation scheme, unless $\mathcal{P} = \mathcal{NP}$. Exact methods, such as branch-and-bound algorithms (Mascis & Pacciarelli, 2002; AitZai, Benmedjdoub, & Boudhar, 2016), integer linear programming (Vermeulen, Hoogeveen, & van den Akker, 2011), constraint programming (Vermeulen et al., 2011) and dynamic programming (Ozolins, 2018) only can solve small problems to optimality within a reasonable time. On account of the complexity of the $J_m|nwt|C_{max}$ problem, the majority of relevant studies seek to develop approximate algorithms that can obtain near-optimal solutions to the problem in a reasonable time. These approximate algorithms usually decompose the problem into two sub-problems—the sequencing and the timetabling problems. The sequencing problem is to find an optimal processing sequence of jobs, whereas the timetabling problem is to find optimal starting times for the jobs in a given processing sequence that yield the minimum makespan.

The majority of the approximate algorithms in the literature solve the $J_m|nwt|C_{max}$ problem by using meta-heuristic algorithms. For

* Corresponding author at: Department of Information Management, Chang Gung University, Taoyuan, Taiwan.

E-mail address: swlin@mail.cgu.edu.tw (S.-W. Lin).

<https://doi.org/10.1016/j.cie.2020.106615>

Received 16 July 2019; Received in revised form 14 June 2020; Accepted 16 June 2020

Available online 27 June 2020

0360-8352/ © 2020 Elsevier Ltd. All rights reserved.

example, Brizuela, Zhao, and Sannomiya (2001) proposed a genetic algorithm (GA) with a simple decoding technique to solve the $J_m|nwt|C_{\max}$ problem. Experimental results revealed that GA easily solved problems that were larger than usual. Later, Schuster and Framinan (2003) used a fast deterministic variable neighborhood search algorithm (VNS) and a hybridization of GA with simulated annealing (GASA) to improve the convergence of the solutions. Their computational results demonstrated that VNS and GASA algorithms compared extremely favorably to the results in the literature. Schuster (2006) presented some new theoretical properties related to the complexity of the two sub-problems of $J_m|nwt|C_{\max}$, and showed both the sequencing and the timetabling problems were $\mathcal{NP}$ -hard in the strong sense. He presented a fast Tabu search (FTS) algorithm for solving the $J_m|nwt|C_{\max}$ problem that compared extremely favorably to the best existing algorithms.

In the same year, Framinan and Schuster (2006) proposed a new timetabling procedure, and embedded it into a meta-heuristic named “complete local search with memory” (CLM) for solving the $J_m|nwt|C_{\max}$ problem. Comparing the computational results of CLM with those of existing meta-heuristics confirmed the effectiveness of the CLM algorithm. Zhu, Li, and Wang (2009) developed the complete local search with limited memory (CLLM) algorithm with an integrated shift timetabling procedure to solve efficiently the $J_m|nwt|C_{\max}$ problem. Their experimental results demonstrated that CLLM outperformed VNS (Schuster & Framinan, 2003), GASA (Schuster & Framinan, 2003) and CLM (Framinan & Schuster, 2006) and required only slightly more computation time.

Bożejko and Makuchowski (2009) proposed a hybrid Tabu search (HTS) algorithm to solve this complex problem. A comparative analysis revealed that HTS found much better solutions than GASA (Schuster & Framinan, 2003) and FTS (Schuster, 2006) algorithms in a comparable time. Pan and Huang (2009) presented a hybrid genetic algorithm (HGA) to improve the efficiency of the traditional GA for solving the $J_m|nwt|C_{\max}$ problem. The computational results showed that HGA accelerated convergence and outperformed GA for large test instances. Also, Santosa, Budiman, and Wiratno (2011) proposed a hybrid meta-heuristic named cross entropy with genetic algorithm (CEGA). Their results demonstrated that CEGA provided better or at least as good makespans than those provided by GASA (Schuster & Framinan, 2003) and HTS (Bożejko & Makuchowski, 2009), especially for small test instances. Bożejko and Makuchowski (2011) presented a GA with an automatic adjustment mechanism. They showed that this algorithm outperformed the GASA of Schuster and Framinan (2003).

Zhu and Li (2012) remodeled the $J_m|nwt|C_{\max}$ problem by introducing time difference, and presented a modified complete local search with memory (MCLM) that used an efficient shift penalty-based timetabling method. Experimental results revealed that MCLM outperformed CLM (Framinan & Schuster, 2006) and CLLM (Zhu et al., 2009). Li, Xu, and Li (2016) solved the $J_m|nwt|C_{\max}$ problem using a complete local search with a memory and variable neighborhood structure (CLMMV) that integrated an alternative delay timetabling method. Their analytical results indicated that CLMMV could find similar solutions to CLLM (Zhu et al., 2009) in shorter computation times. AitZai et al. (2016) adapted the particle swarm optimization (PSO) algorithm and obtained better results than the heuristics of Mascis and Pacciarelli (2002).

Sundar et al. (2017) proposed a hybrid artificial bee colony (HABC) algorithm that integrated an artificial bee colony algorithm with a local search to find high-quality solutions to the $J_m|nwt|C_{\max}$ problem. They investigated its performance on standard benchmark instances of the problem and found that HABC outperformed the best two meta-heuristics—CLLM (Zhu et al., 2009) and MCLM (Zhu and Li (2012)—in the literature. Valenzuela, Brizuela, Cosío-León, and Romero-Ocaño (2019) presented a genetic algorithm with the mixed standard and reverse timetabling method (*mix-D/GA*), which can find better solutions than when using the direct or the reverse timetabling separately for solving

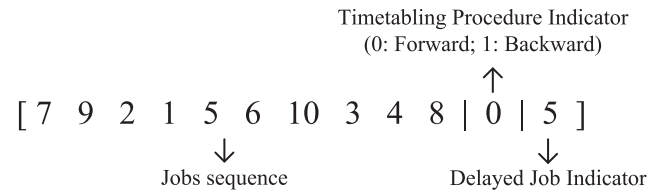


Fig. 1. Example of a solution representation.

Table 1

An example of 4 Jobs * 4 Machines.

Job	Machine sequence (time)
J_1	4 (7) → 3 (9) → 1 (8) → 2 (4)
J_2	1 (4) → 4 (8) → 2 (2) → 3 (4)
J_3	3 (5) → 2 (7) → 1 (1) → 4 (4)
J_4	4 (3) → 2 (1) → 1 (2) → 3 (3)

the $J_m|nwt|C_{\max}$ problem. Experimental results on a set of benchmarks revealed that the *mix-D/GA* algorithm produced competitive results with those produced by CLLM, MCLM and HABC. To the best of our knowledge, in the existing literature, CLLM, MCLM and HABC are by far the best meta-heuristics for solving the $J_m|nwt|C_{\max}$ problem. Accordingly, this study uses them as benchmark algorithms to evaluate the performance of the meta-heuristic that is proposed herein.

The rest of this paper is organized as follows: Section 2 formally defines the $J_m|nwt|C_{\max}$ problem of interest. Section 3 describes the proposed multi-start simulated annealing with bi-directional shift timetabling (MSA-BST) algorithm. Section 4 provides a comparative analysis of the results that are obtained using the proposed MSA-BST algorithm with those yielded by the best-to-date meta-heuristic algorithm. Finally, Section 5 provides a general conclusion and recommendations for future research.

2. Problem definition

The $J_m|nwt|C_{\max}$ problem involves a finite set J of independent jobs that have to be processed on a set M of machines. Each job j ($j \in J$) has its own processing order $\{O_{j1}, O_{j2}, \dots, O_{jn_j}\}$, where n_j is the total number of operations in job j . Each operation must be processed on a specified machine. Operation O_{jk} ($j \in J$; $k = 1, 2, \dots, n_j$) of job j requires the exclusive use of machine M_l ($M_l \in M$), on which it must be processed without interruption for P_{jk} time units. Every job must be processed without waiting between the finalization of one operation and the beginning of the next. Hence, once the first operation of any job is begun, the job must be processed operation by operation until all of its operations have been completed. If necessary, the starting time of the first operation must be postponed so that each job can be processed continuously from its start to its completion. The following critical assumptions are made in the addressed $J_m|nwt|C_{\max}$ problem.

- Each job has only one feasible sequence of operations and each operation must be performed on a single machine.
- No machine can be used more than once in a job.
- All jobs are available at the beginning of the planning horizon.
- All machines are available throughout the planning horizon.
- All machines are different and their processing speeds are constant.
- No machine may process more than one operation at once.
- No job may be processed by more than one machine at the same time.
- The operations of different jobs have no precedence constraints.
- The processing time may vary among operations, but it is known and fixed for any given operation.

Given all of the above constraints, the objective is to find a feasible

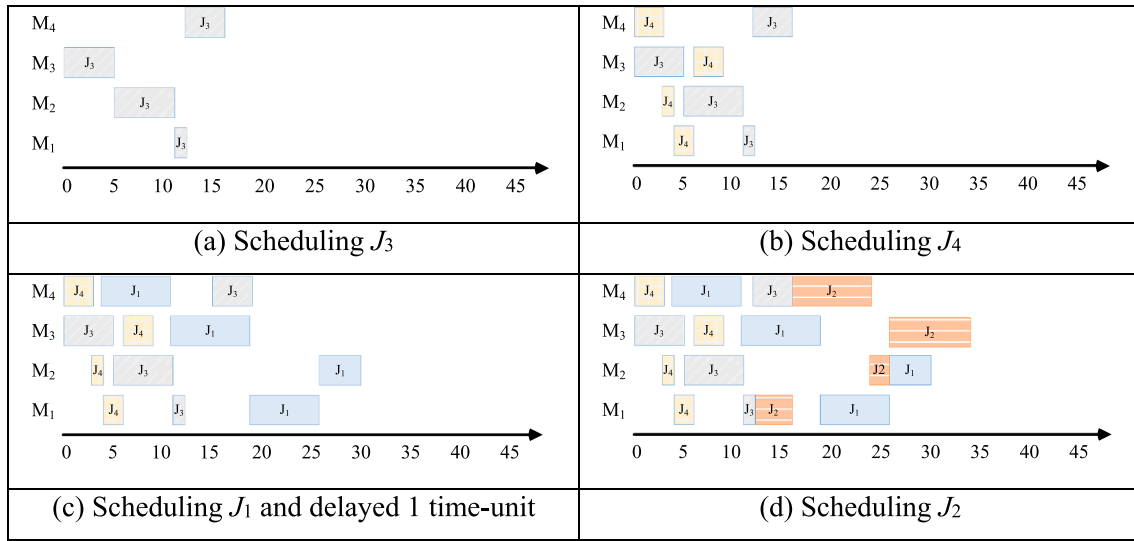


Fig. 2. The optimal solution for the example, obtained by applying the bi-directional shift timetabling procedure.

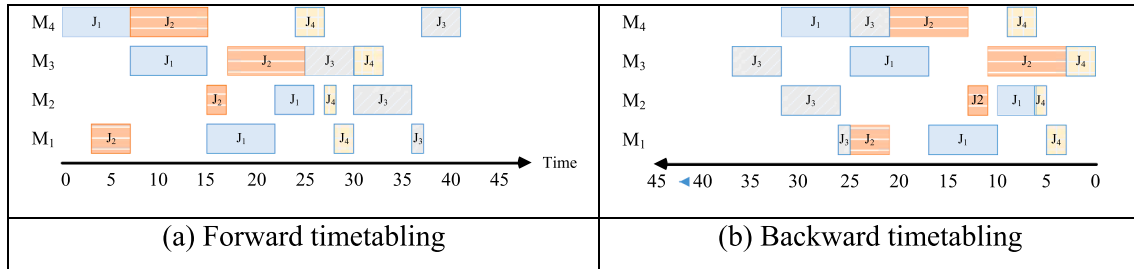


Fig. 3. The best solutions for the example, obtained by applying the forward and backward non-delay timetabling procedures, respectively.

timetable schedule for all jobs that minimizes the makespan $C_{\max} = \max\{C_j\}$, where C_j denotes the completion time of the final operation of job j . Let $S_{jk}(j \in \mathbf{J}; k = 1, 2, \dots, n_j)$ be the starting time of operation O_{jk} , and M be a sufficiently large positive number. Based on the MILP model of Brizuela et al. (2001), the $J_m | nwt | C_{\max}$ problem considered herein can be formally specified as follows.

$$\text{Minimize } C_{\max} = \max_{j \in \mathbf{J}} \{S_{jn_j} + P_{jn_j}\} \quad (1)$$

subject to

$$S_{jk} + P_{jk} = S_{j(k+1)}, \quad \forall k = 1, 2, \dots, n_j - 1, j \in \mathbf{J}, \quad (2)$$

$$S_{ji} + P_{ji} \leq S_{j'i} + My, \quad \forall j, j' \in \mathbf{J}, i \in \mathbf{M}, \quad (3)$$

$$S_{j'i} + P_{j'i} \leq S_{ji} + M(1 - y), \quad \forall j, j' \in \mathbf{J}, i \in \mathbf{M}, \quad (4)$$

$$y \in \{0, 1\}; S_{jk} \geq 0, \quad k = 1, 2, \dots, n_j, j \in \mathbf{J}. \quad (5)$$

Constraint (1) defines the makespan of the schedule. Constraint set (2) is no-wait constraints on consecutive operations. Constraint sets (3) and (4) guarantee that every job is assigned to one position in the sequence of jobs. Constraint set (5) defines the domain of variable y and the ranges of the starting times of all operations.

3. Proposed multi-start simulated annealing with bi-directional shift timetabling algorithm

Based on a concept of Metropolis, Rosenbluth, Rosenbluth, Teller, and Teller (1953) that was inspired by annealing in metallurgy, Kirkpatrick et al. (1983) proposed the classical simulated annealing (SA) algorithm. The procedures of executing the classical SA algorithm usually begin with a randomly generated initial solution. At each

iteration, a neighborhood solution is generated from the incumbent solution, and then a decision is made probabilistically to replace or not to replace the incumbent solution with the new one, from which the search process will continue. Typically, this step is repeated until a pre-specified computation budget has been exhausted. In this study, an effective multi-start simulated annealing with bi-directional shift timetabling (MSA-BST) algorithm, which integrates the advantages of the SA algorithm, the multi-start hill climbing strategy and a bi-directional shift timetabling procedure, is proposed for solving the $J_m | nwt | C_{\max}$ problem. The following subsections discuss the multi-start initialization, solution representation, bi-directional shift timetabling procedure, neighborhoods, and details of the MSA-BST procedure.

3.1. Multi-start initialization and solution representation

In this work, the initial solution set $P = \{\pi_1, \dots, \pi_p\}$ is generated randomly, where p denotes the number of solutions in P . A solution π_q ($q = 1, \dots, p$) is represented by a string of numbers consisting of three segments, which have n , 1 and 1 element(s), respectively. The element(s) in the first, second and third segments correspond to the sequence of n jobs, the timetabling procedure used, and the selected job to be delayed, respectively. In the first segment, the random permutation of n jobs is generated using the Fisher-Yates shuffle (Fisher & Yates, 1948). Any permutation of n jobs will be produced by this algorithm with a probability of exactly $1/n!$, thus yielding a uniform distribution over all such permutations. In the second segment, the random binary variable, *direction*, is generated to indicate which timetabling procedure will be used to decode the sequence. If *direction* = 0, then the forward timetabling procedure will be used; otherwise, the backward timetabling procedure will be used. The forward and backward timetabling procedures are described in section 3.2. The motivation for embedding the bi-

```

MSA-BST (  $p, T_0, I_{iter}, \alpha, N_{non-improving}$  )
Step 1: Set  $T = T_0$ ;  $R = 0$ ;  $N = 0$ ;
Step 2: Generate the initial population  $P = \{\pi_1, \dots, \pi_p\}$ , and let  $\pi_{best}$  be the best solution
in  $P$ ,  $F_{best} = C_{max}(\pi_{best})$ ;
Step 3: For  $j = 1$  to  $I_{iter}$  {
    Step 3.1: Randomly select a solution  $\pi_q$  from  $P$ ;
    Step 3.2: Generate a neighborhood solution  $\pi'_q \in N(\pi_q)$  as follows:
        Generate random numbers  $r_1, r_2, r_3 \in U(0, 1)$ ;
        If ( $r_1 \leq 0.2$ ), generate  $\pi'_q$  by swap operator;
        Else If ( $0.2 < r_1 \leq 0.4$ ), generate  $\pi'_q$  by insertion operator;
        Else If ( $0.4 < r_1 \leq 0.6$ ), generate  $\pi'_q$  by 2-swap operator;
        Else If ( $0.6 < r_1 \leq 0.8$ ), generate  $\pi'_q$  by 2-insertion operator;
        Else, generate  $\pi'_q$  by inversion operator;
        If ( $r_2 \leq 0.01$ ), change the direction of  $\pi'_q$ ;
        If ( $r_3 \leq 0.01$ ), change the  $j_{delay}$  of  $\pi'_q$ ;
    Step 3.3: If  $\Delta_q = C_{max}(\pi'_q) - C_{max}(\pi_q) \leq 0$  {Let  $\pi_q = \pi'_q$ ; }
    Else {
        Generate a random number  $r \in U(0, 1)$ ;
        If  $r < \exp(-\Delta_q / T)$  { Let  $\pi_q = \pi'_q$ ; }
        Else {Discard  $\pi'_q$ ; }
    }
    Step 3.4: If  $C_{max}(\pi_q) < F_{best}$  {
        Perform local search for  $\pi_q$ ;
         $\pi_{best} = \pi_q$ ;  $F_{best} = C_{max}(\pi_q)$ ;  $R = 0$ ; Let  $\pi_q = \pi_{best}$  ( $q = 1, \dots, p$ );
    }
}
Step 4: If  $N = I_{iter}$  {  $T = T \times \alpha$ ;  $N = 0$ ;  $R = R + 1$ ; }
Else {Go to Step 3;}
Step 5: If  $R = N_{non-improving}$  {Terminate the PSA procedure;}
Else {Go to Step 3;}

```

Fig. 4. Pseudo-code of the proposed MSA-BST algorithm.

directional shift timetabling procedure into the proposed algorithm is to enlarge the search space, and thus find better solutions within the given processing sequences. In the third segment, a random variable, j_{delay} , varying from 1 to $4n$, is generated to indicate the delayed job. If $j_{delay} \in [1, n]$, then the earliest possible processing time of the first operation of job j_{delay} will be delayed; otherwise, no operation of any job in the sequence will be delayed. Therefore, the probability that a selected job will be delayed is 25%. Due to the complexity of the shift timetabling method, calculating delay timetables consumes more computing time as the range of delay times or the problem size n increase. To trade-off solution quality and computing time in this study, the first operation of the selected job is delayed for one unit of time. Fig. 1 displays an example of the solution representation. The permutation list of ten jobs in the first segment reveals that these jobs are to be processed in the sequence 7-9-2-1-5-6-10-3-4-8; In the second segment,

direction = 0 means that the forward timetabling procedure will be used to decode the sequence; In the third segment, $j_{delay} = 5$ indicates that the starting processing time of the first operation of job 5 will be delayed for one unit of time.

3.2. Bi-directional shift timetabling procedure

In this study, a bi-directional (forward and backward) shift timetabling procedure is used to decode the starting time of each job in a given processing sequence. The proposed bi-directional shift timetabling procedure is briefly described as follows.

Step 1. Set the idle time interval of each machine as $[0, \infty)$.

Step 2. Sequentially pick jobs from the job sequence as specified by the indicative variable *direction*=0 or 1, until all jobs in the sequence

```

Bestf =  $C_{\max}(\pi_{best})$ ;
For  $i = 1$  to  $n - 1$ 
{
  For  $j = i + 1$  to  $n$ 
  {
     $\pi_{temp}$  = swap the  $i^{th}$  and the  $j^{th}$  elements of  $\pi_{best}$ ;
    If  $Bestf < C_{\max}(\pi_{temp})$ 
    {
       $Bestf = C_{\max}(\pi_{temp})$ ;
       $\pi_{best} = \pi_{temp}$ ;
    }
  }
}

```

Fig. 5. Pseudo-code of the local search procedure.

Table 2
Orthogonal array and the obtained RPDs.

Experiment No.	p	T_0	I_{iter}	λ	$N_{non-improving}$	RPD
1	3	45	6000n*	0.96	0.6n	2.4042
2	3	50	8000n	0.97	0.8n	1.8656
3	3	55	10000n	0.98	1.0n	1.6923
4	3	60	12000n	0.99	1.2n	1.8268
5	5	45	8000n	0.98	1.2n	1.5534
6	5	50	6000n	0.99	1.0n	2.1590
7	5	55	12000n	0.96	0.8n	1.5279
8	5	60	10000n	0.97	0.6n	2.1525
9	7	45	10000n	0.99	0.8n	2.1198
10	7	50	12000n	0.98	0.6n	2.0658
11	7	55	6000n	0.97	1.2n	1.6905
12	7	60	8000n	0.96	1.0n	1.6211
13	9	45	12000n	0.97	1.0n	1.3476
14	9	50	10000n	0.96	1.2n	1.3063
15	9	55	8000n	0.99	0.6n	2.7474
16	9	60	6000n	0.98	0.8n	2.4359

* n denotes the number of jobs.

Table 3
ARPDs obtained by different levels of each parameter.

Level	p	T_0	I_{iter}	λ	$N_{non-improving}$
1	1.9472	1.8563	2.1724	1.7149	2.3425
2	1.8482	1.8492	1.9469	1.7640	1.9873
3	1.8743	1.9145	1.8177	1.9369	1.7050
4	1.9593	2.0091	1.6920	2.2133	1.5943
Range	0.1111	0.1599	0.4804	0.4984	0.7482
Rank	5	4	3	2	1

have been selected. If $direction=0$, then the forward timetabling procedure is used to decode the starting times of operations from the left of the job sequence to the right, and the jobs in the sequence are processed one by one, as determined by their operation sequences on the machines. Otherwise, the backward timetabling procedure is used to decode the starting times of operations from the right of the job sequence to the left, and the jobs in the sequence are processed one by one, as determined by the reverse of their operation sequences on the machines. Find the earliest starting times of all operations of the picked job that satisfy the delay requirement indicated by the variable j_{delay} , the idle time intervals of the machines and the processing times of all operations of the picked job without violating the no-wait constraint. If the processing times of any operations of the picked job do not satisfy the idle time intervals of the corresponding machines, then the starting time of the first operation

of the picked job is delayed until the processing times of all of its operations fall within the idle time intervals of their corresponding machines without violating the no-wait constraint. The idle time intervals of every machine are updated after the starting time of the picked job is set.

The bi-directional shift timetabling procedure can be illustrated by applying it to the data in Table 1. It is assumed that the permutation list obtained by the proposed MSA-BST is (3 4 1 2 0 1), which means the job processing sequence is 3–4–1–2. In addition, $direction = 0$ means that the forward timetabling procedure is used to decode the sequence, and $j_{delay} = 1$ indicates that the starting processing time of the first operation of Job 1 will be delayed for one unit of time. From the Gantt chart shown in Fig. 2, the makespan of the decoded sequence is 34, which is the optimal solution, verified by implementing the MILP model. It can be found that if the starting processing time of the first operation of Job 1 is not delayed by 1 time-unit, the starting processing time of the third operation of Job 2 will be delayed, and thus the makespan will increase. Furthermore, all possible job sequences in the example are enumerated by applying the forward and backward non-delay timetabling procedures, and their best solutions, (1 2 3 4) and (3 1 2 4), respectively are obtained. As shown in Fig. 3, the best makespan values obtained by applying the forward and backward non-delay timetabling procedures are 41 and 37, respectively, which is larger than the optimal solution obtained by applying the bi-directional shift timetabling procedure. It is intuitively quite clear that, in some cases, the bi-directional shift timetabling procedure allows the exploitation of new schedules that cannot be obtained by the non-delay timetabling methods for the problem, and it can thus find better starting times for jobs in order to realize the minimum makespan within the given processing sequence.

Note that the proposed bi-directional shift timetabling method differs from the timetabling methods used in CLM (Framinan & Schuster, 2006), CLLM (Zhu et al., 2009), MCLM (Zhu & Li, 2012), CLMMV (Li et al., 2016), and mix-D/GA (Valenzuela et al., 2019). The inverse timetabling used in CLM is a non-delay timetabling obtained by the inverse processing sequence of the inverse instance. The shift timetabling method used in CLLM generates a timetable by performing right shift or left shift strategies on each job. The shift penalty-based timetabling method used in MCLM constructs two initial timetables from time difference-based sets, and improves them by an investigated timetable tightening method. By alternatively adopting the “as early as possible” or the “as late as possible” rules, the improved non-delay timetabling method used in CLMMV is identical to non-delay timetabling, with the exception that it does not include the ordinal constraint, i.e., the starting time of the successor job cannot be less than its predecessor job for a given processing sequence. The mixed standard and reverse timetabling methods used in the mix-D/GA algorithm do not include the shift timetabling mechanism. Moreover, the reverse timetabling of the mix-D/GA algorithm decodes the order of operations in a completely reversed sequence, and then re-calculates the start and end times for all operations of a given job. That is, the reverse timetabling is not equivalent to the backward timetabling procedure described in this paper.

3.3. Neighborhoods

Let $N(\pi_q)$ represent the sets of solutions that neighbor the solution π_q ($q = 1, \dots, p$), which can be generated using swap, insertion, 2-swap, 2-insertion, or inversion operators (Chen, 2006; Bozejko & Wodecki, 2006; Lin & Ying, 2015). The swap operator randomly selects two jobs in π_q and switches their positions. The insertion operator randomly selects one job in π_q and inserts it in the position immediately before another randomly selected job in π_q . The 2-swap and 2-insertion operators execute swap and insertion operations twice, respectively. The inversion operator randomly selects a substring in π_q and reverses its order.

Table 4
Comparison of results obtained by compared algorithms (small-sized test problems).

Problem	(n, m)	BKS	CLLM			MCLM			HABC				MSA-BST				TEM_F		TEM_B	
			BRPD	APRD	ACT	BRPD	APRD	ACT	C _{max}	BRPD	APRD	ACT	C _{max}	BRPD	APRD	ACT	C _{max}	RPD	C _{max}	RPD
Ft06	(6, 6)	73	0.00	0.00	0.00	0.00	0.00	0.00	73	0.00	0.00	0.11	73	0.00	0.00	0.54	73	0.00	73	0.00
La01	(10, 5)	971	0.00	0.10	9.83	0.00	0.02	3.19	975	0.41	0.41	0.55	971	0.00	0.00	2.00	975	0.41	971	0.00
La02	(10, 5)	937	0.00	0.21	23.17	0.00	0.00	5.34	961	2.56	2.56	0.99	937	0.00	0.00	1.82	961	2.56	937	0.00
La03	(10, 5)	820	0.00	0.00	8.43	0.00	0.00	2.18	820	0.00	0.00	0.49	820	0.00	0.00	1.80	820	0.00	820	0.00
La04	(10, 5)	887	0.00	0.00	23.87	0.00	0.00	4.39	887	0.00	0.00	0.61	887	0.00	0.00	1.71	887	0.00	887	0.00
La05	(10, 5)	777	0.00	0.50	5.62	0.51	0.90	2.74	781	0.51	0.51	0.80	777	0.00	0.00	1.56	781	0.51	777	0.00
Ft10	(10, 10)	1607	0.00	0.00	64.60	0.00	0.00	5.51	1607	0.00	0.00	6.92	1607	0.00	0.00	3.46	1607	0.00	1609	0.12
Orb01	(10, 10)	1615	0.00	0.00	50.55	0.00	0.00	4.67	1615	0.00	0.00	5.62	1615	0.00	0.00	3.37	1615	0.00	1626	0.68
Orb02	(10, 10)	1485	0.00	1.89	33.70	2.16	2.16	4.70	1485	0.00	0.00	4.82	1485	0.00	0.22	3.78	1485	0.00	1518	2.22
Orb03	(10, 10)	1599	0.00	0.00	49.85	0.00	0.00	9.65	1599	0.00	0.00	7.07	1599	0.00	0.00	3.44	1599	0.00	1599	0.00
Orb04	(10, 10)	1653	0.00	0.42	28.09	0.00	0.12	5.51	1653	0.00	0.00	3.71	1653	0.00	0.00	3.88	1653	0.00	1653	0.00
Orb05	(10, 10)	1365	0.15	0.15	9.83	0.00	0.00	5.97	1370	0.37	0.37	3.35	1365	0.00	0.14	3.75	1370	0.37	1367	0.15
Orb06	(10, 10)	1555	0.00	0.00	30.89	0.00	0.00	2.49	1555	0.00	0.00	4.04	1555	0.00	0.00	3.62	1555	0.00	1557	0.13
Orb08	(10, 10)	1319	0.00	0.00	42.13	0.00	0.00	4.49	1319	0.00	0.00	6.31	1319	0.00	0.00	3.20	1319	0.00	1319	0.00
Orb09	(10, 10)	1445	0.00	0.83	13.34	0.00	0.00	2.98	1445	0.00	0.28	2.88	1445	0.00	0.00	3.59	1445	0.00	1445	0.00
Orb10	(10, 10)	1557	0.00	0.00	48.45	0.00	0.00	8.32	1557	0.00	0.00	3.87	1557	0.00	0.09	4.36	1557	0.00	1564	0.45
La16	(10, 10)	1575	0.00	0.00	28.79	1.84	1.84	3.97	1575	0.00	0.00	3.81	1575	0.00	0.00	3.39	1575	0.00	1575	0.00
La17	(10, 10)	1371	0.00	0.22	37.21	0.00	0.12	8.32	1384	0.95	0.95	3.24	1371	0.00	0.00	3.56	1384	0.95	1371	0.00
La18	(10, 10)	1417	0.00	1.13	35.81	2.82	2.82	4.32	1417	0.00	5.22	5.01	1417	0.00	0.79	3.97	1417	0.00	1417	0.00
La19	(10, 10)	1482	0.00	0.14	11.23	0.00	0.61	3.51	1482	0.00	0.43	2.42	1482	0.00	0.49	3.78	1482	0.00	1491	0.61
La20	(10, 10)	1526	0.00	0.13	4.21	1.31	1.31	3.79	1526	0.00	0.00	2.11	1526	0.00	0.00	3.45	1526	0.00	1580	3.54
Average			0.01	0.27	26.65	0.41	0.47	4.57		0.23	0.51	3.27		0.00	0.08	3.13		0.23		0.38

3.4. Procedures of MSA-BST

Fig. 4 shows the procedure for implementing the proposed MSA-BST algorithm. First, after the parameter values have been set, the initial solution set $P = \{\pi_1, \dots, \pi_p\}$ is generated randomly. At each iteration, a solution π_q is randomly selected from P , and a neighborhood solution $\pi'_q \in N(\pi_q)$ is generated by applying a swap, insertion, 2-swap, 2-inversion, or inversion operator. Since there are a very large number of combinations of these operators, Step 3.2 fixes the probability of performing any one of the operators at 0.2 in order to simplify the parameters settings. After one of the five operators is performed, there is a 0.01 probability to change the *direction* of π'_q and 0.01 probability to change the j_{delay} of π'_q . Let $C_{\max}(\pi_q)$ and $C_{\max}(\pi'_q)$ be the makespan of π_q and π'_q , respectively, and let $\Delta_q = C_{\max}(\pi'_q) - C_{\max}(\pi_q)$. The probability of replacing π_q with π'_q , given that $\Delta_q > 0$, is $\exp(-\Delta_q/T)$, where T is the current temperature. This replacement is performed by generating a random number $r \in U(0, 1)$, and replacing solution π_q with π'_q if $r < \exp(-\Delta_q/T)$. If $\Delta_q \leq 0$, the probability of replacing π_q with π'_q is 1.0.

If π'_q is better than the incumbent best solution π_{best} found so far, it will replace π_{best} and a local search (LS) procedure, as depicted in Fig. 5, will be performed. As shown in Fig. 5, the LS procedure improves π_{best} by iteratively exchanging jobs in the solution with jobs positioned after them. T is reduced after I_{iter} iterations from the last reduction, using the formula $T \leftarrow \alpha T$, where $0 < \alpha < 1$. If the incumbent best solution, π_{best} , is not improved in $N_{non-improving}$ successive temperature reductions, then the algorithm is terminated. Upon the termination of the MSA-BST procedure, the (near) global optimal schedule is obtained.

4. Experimental results

This section discusses the computational results that are obtained by applying the proposed MSA-BST algorithm and three other algorithms for comparison—CLLM (Zhu et al., 2009), MCLM (Zhu & Li, 2012), and HABC (Sundar et al., 2017)—to solve the $J_m|nwtlC_{\max}$. To verify the performance of the bi-directional shift timetabling procedure, its computational results are compared with those obtained using the total enumeration method with forward and backward decoding, denoted as TEM_F and TEM_B, respectively. The proposed methods are implemented in C++ on an Intel® Core™ i7-920 processor that runs at

2.67 GHz and with 20 GB of RAM. The following subsections discuss the test problems, parameter selection, and experimental results.

4.1. Test problems

Sundar et al. (2017) used the classical job-shop standard benchmark problem instances (Beasley, 1990) with no-wait constraints to evaluate the performance of CLLM, MCLM and HABC. These instances are used herein to verify the performance of the proposed MSA-BST. They are grouped into sets of small and large problem instances that have 21 and 40 test instances, respectively. For the small problem instances, the number of jobs varies from six to ten and the number of machines varies from five to ten. For the large problem instances, the number of jobs varies from 15 to 30 and the number of machines varies from five to 15.

4.2. Parameters selection

The five key parameters of the proposed MSA-BST are p , T_0 , I_{iter} , λ and $N_{non-improving}$, where p is the initial solution set size; T_0 denotes the initial temperature; I_{iter} represents the number of iterations at a particular temperature; λ represents the cooling rate, and $N_{non-improving}$ is the maximum number of consecutive temperature reductions over which the best value of the objective function that has been obtained so far has not been improved. As shown in Table 2, each of the five parameters has four levels so the number of possible combinations of parameter values is 4^5 . In order to more effectively determine the best combination of the five parameter values, Type B of the Taguchi L16 orthogonal experiment design, which involves 16 parameter combinations (see <https://www.york.ac.uk/depts/maths/tables/l16b.htm>), is applied to five randomly selected test instances (La08, LA15, LA23, Swv04 and La36). For each parameter combination, the proposed MSA-BST algorithm is executed independently 30 times to each of five selected instances. Tables 2 and 3 present the average relative percentage deviation (ARPD) from the best solution as the response variable.

Table 3 presents the significance value of each parameter. The most significant of the five parameters is $N_{non-improving}$, which has the largest range of ARPD. According to the experimental results, the quality of the solution increases with the computing time that is used to execute the proposed MSA-BST algorithm. To balance computational time with solution quality, p_{size} , T_0 , I_{iter} , λ and $N_{non-improving}$ are set to 5, 50.0,

Table 5
Comparison of results obtained by compared algorithms (large-sized test problems).

Problem	(n, m)	BKS	CLLM			MCLM			HABC				MSA-BST			
			BRPD	APRD	ACT	BRPD	APRD	ACT	C _{max}	BRPD	APRD	ACT	C _{max}	BRPD	APRD	ACT
La06	(15, 5)	1248	0.00	2.80	185	0.00	0.75	63	1248	0.00	0.00	28	1248	0.00	0.00	49
La07	(15, 5)	1172	0.00	2.47	153	0.51	2.46	58	1172	0.00	0.92	63	1172	0.00	0.00	47
La08	(15, 5)	1244	0.00	2.25	176	0.00	0.47	56	1244	0.00	0.15	60	1244	0.00	1.03	65
La09	(15, 5)	1358	0.00	1.47	163	0.52	0.73	74	1362	0.29	0.67	24	1358	0.00	0.46	51
La10	(15, 5)	1287	0.00	0.70	141	0.00	0.04	48	1294	0.54	0.70	46	1287	0.00	0.22	57
La11	(20, 5)	1621	3.08	5.86	314	0.87	2.49	308	1627	0.37	1.05	182	1621	0.00	1.53	191
La12	(20, 5)	1425	1.89	5.69	350	0.28	2.48	416	1434	0.63	1.89	119	1425	0.00	2.26	184
La13	(20, 5)	1580	2.78	5.13	449	1.58	2.63	213	1580	0.00	1.27	156	1598	1.14	2.73	199
La14	(20, 5)	1610	5.03	6.89	326	2.36	3.81	220	1640	1.86	2.68	110	1610	0.00	1.67	167
La15	(20, 5)	1679	0.89	4.17	340	0.36	1.99	298	1679	0.00	0.80	169	1685	0.36	1.20	206
La21	(15, 10)	2030	0.89	3.64	215	0.89	1.00	55	2043	0.64	1.16	50	2030	0.00	0.52	110
La22	(15, 10)	1852	1.89	3.24	249	2.70	2.90	100	1852	0.00	0.75	64	1852	0.00	1.11	111
La23	(15, 10)	2021	0.54	3.81	216	0.05	2.02	35	2032	0.54	1.26	85	2021	0.00	1.43	129
La24	(15, 10)	1972	2.18	3.86	296	2.18	2.97	69	1994	1.12	2.16	69	1972	0.00	1.89	110
La25	(15, 10)	1906	0.58	3.41	209	1.26	2.66	50	1906	0.00	0.00	65	1906	0.00	0.49	120
La26	(20, 10)	2506	1.88	8.02	570	1.04	3.82	245	2506	0.00	2.18	157	2506	0.00	1.60	315
La27	(20, 10)	2666	3.04	6.45	586	1.83	3.33	272	2674	0.30	0.34	109	2666	0.00	0.49	289
La28	(20, 10)	2552	2.82	7.84	569	0.31	4.64	220	2560	0.31	3.46	139	2552	0.00	3.03	332
La29	(20, 10)	2300	8.22	10.39	546	2.91	5.53	312	2389	3.87	5.30	373	2300	0.00	3.42	332
La30	(20, 10)	2452	8.69	11.87	577	3.75	7.05	264	2452	0.00	5.43	174	2452	0.00	5.32	338
La31	(30, 10)	3369	11.16	15.28	1817	6.11	9.60	2176	3592	6.62	8.27	1205	3369	0.00	6.11	1316
La32	(30, 10)	3835	5.03	11.06	1894	0.00	5.62	2327	3913	2.03	4.66	1117	3859	0.63	3.35	1304
La33	(30, 10)	3413	9.84	12.57	1816	4.71	8.18	2109	3529	3.40	6.14	1084	3413	0.00	3.93	1352
La34	(30, 10)	3551	7.69	10.72	1934	3.75	6.74	2370	3610	1.66	4.72	987	3551	0.00	3.17	1276
La35	(30, 10)	3556	5.74	12.04	1836	3.99	7.08	2165	3593	1.04	5.30	1262	3556	0.00	3.42	1562
La36	(15, 15)	2685	0.00	3.46	376	1.90	4.98	133	2685	0.00	0.29	54	2685	0.00	0.50	166
La37	(15, 15)	2831	4.63	6.64	355	4.63	4.74	65	2938	3.78	5.65	129	2831	0.00	4.65	179
La38	(15, 15)	2525	3.64	5.98	349	0.00	1.85	64	2525	0.00	2.52	219	2525	0.00	2.31	159
La39	(15, 15)	2697	0.00	2.93	392	1.19	2.43	81	2703	0.22	0.88	103	2697	0.00	0.37	160
La40	(15, 15)	2580	0.54	5.00	199	0.00	0.00	43	2594	0.54	0.54	134	2580	0.00	0.52	135
Swv01	(20, 10)	2318	0.43	3.06	450	0.65	0.80	362	2318	0.00	0.41	614	2328	0.43	0.80	376
Swv02	(20, 10)	2417	0.04	3.14	546	0.04	0.42	343	2417	0.00	0.02	675	2417	0.00	0.00	358
Swv03	(20, 10)	2381	1.43	4.29	532	0.00	1.01	363	2381	0.00	0.49	715	2381	0.00	1.01	448
Swv04	(20, 10)	2462	3.25	4.07	474	0.00	2.09	299	2506	1.79	2.08	1214	2463	0.04	1.94	391
Swv05	(20, 10)	2333	0.00	6.94	500	0.00	0.00	200	2333	0.00	0.00	376	2333	0.00	1.72	371
Swv06	(20, 15)	3278	2.99	5.46	798	0.40	2.09	525	3291	0.40	0.80	622	3278	0.00	0.91	562
Swv07	(20, 15)	3188	2.60	4.17	826	1.63	0.97	410	3188	0.00	0.00	582	3188	0.00	0.28	564
Swv08	(20, 15)	3423	3.13	6.17	807	0.00	0.85	290	3423	0.00	1.29	808	3423	0.00	2.09	577
Swv09	(20, 15)	3246	1.88	3.57	739	0.74	1.59	315	3246	0.00	1.00	1075	3246	0.00	0.87	517
Swv10	(20, 15)	3451	1.07	3.28	802	0.00	1.19	365	3462	0.32	0.85	1001	3465	0.41	1.23	474
Average			2.74	5.74	601	1.33	2.90	459		0.81	1.95	405		0.08	1.74	391

*New Best Known Solution best in **bold**.

Table 6
Kruskal-Wallis Test for BRPD and ARPD (small-sized test problems).

	chi-squared	df	p-Value
Test on BRDP	14.562	5	0.012410*
Test on ARDP	15.977	5	0.006909*

* Significant difference exists among the six compared algorithms for the small-sized test problems.

Table 7
Kruskal-Wallis Test for BRPD and ARPD (large-sized test problems).

	chi-squared	df	p-Value
Test on BRDP	48.858	3	< 0.00001*
Test on ARDP	47.890	3	< 0.00001*

* Significant difference exists among the four compared algorithms for the large-sized test problems.

12000n, 0.96 and 1.0n, respectively, for large problem instances, where n denotes the number of jobs. Since the solution spaces of the small problem instances are relative small, the corresponding parameter values are set to those used for the large problem instances, except with $N_{non-improving}$ set to 2.

4.3. Results and discussion

Like CLLM, MCLM and HABC, the proposed MSA-BST algorithm is executed 20 times for each problem instance using various random seeds. Then, the best and average makespan values of each problem instance from 20 runs are compared. Since EM_F and EM_B are deterministic methods, both of them are run only once for each test instance. To verify the performance of the proposed MSA-BST algorithm, the makespan values that are obtained using the four compared algorithms for each benchmark instance are compared using the relative percentage deviation (RPD) as follows.

$$RPD_h = \frac{C_{\max}^h - C_{\max}^B}{C_{\max}^B} \times 100\%$$

where $C_{\max}^h$ and $C_{\max}^B$ are the makespan values that are obtained using algorithm h and the best known solution (BKS), respectively.

Sundar et al. (2017) implemented CLLM, MCLM and HABC on a PC with a Pentium 4 processor, which differed from the processor that was used in this study. Since the computing times vary with hardware, software and programming skill, a completely fair comparison of the compared algorithms' efficiencies cannot be made. The scores that were measured by the CPU benchmarks (<http://www.cpubenchmark.net/singleThread.html>) are used to adjust the computational times of the

Table 8

Results for the test problems of Adams et al. (1988), Yamada and Nakano (1992), and Taillard (1993).

Instance	Size	BKS	M ₀			M ₁			M ₂			M ₃			M ₄		
			Best	Avg	Time	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time	Best	Avg	Time
yn1	20*20	2362	2385	2439.05	1174.6	2385	2434.15	884.1	2362	2443.10	954.7	2385	2421.40	910.5	2390	2432.75	1101.0
yn2	20*20	2312	2312	2388.45	1040.2	2312	2393.10	1016.5	2323	2403.55	1112.5	2349	2418.80	880.3	2323	2418.10	1161.0
yn3	20*20	2299	2299	2380.40	1160.9	2324	2379.20	803.6	2299	2364.40	1064.1	2324	2350.50	906.2	2299	2374.05	954.2
yn4	20*20	2458	2478	2528.50	1129.2	2478	2528.00	963.5	2474	2528.20	1030.3	2458	2516.60	1005.7	2489	2525.55	1090.2
abz5	10*10	2170	2170	2176.75	4.2	2170	2176.75	4.2	2179	2179.30	4.5	2170	2173.60	5.3	2179	2179.00	5.1
abz6	10*10	1718	1718	1734.10	3.8	1718	1731.80	4.0	1741	1741.00	5.6	1718	1732.95	4.4	1719	1739.90	5.6
abz7	20*15	1563	1564	1617.00	789.7	1589	1619.70	805.2	1601	1620.05	824.9	1563	1611.10	847.3	1585	1619.65	865.5
abz8	20*15	1616	1631	1654.15	758.4	1623	1659.30	726.1	1628	1659.65	877.5	1623	1649.30	737.7	1616	1643.95	968.5
abz9	20*15	1562	1562	1615.95	771.7	1562	1610.05	765.0	1566	1605.15	858.1	1596	1617.90	783.7	1582	1616.60	791.2
ta01	15*15	2674	2674	2690.30	257.8	2674	2703.80	190.1	2674	2683.30	238.8	2674	2709.00	211.8	2674	2682.55	220.4
ta02	15*15	2646	2663	2669.70	221.7	2687	2704.30	194.9	2663	2666.10	191.4	2687	2705.30	239.2	2646	2662.00	220.3
ta03	15*15	2600	2600	2609.45	212.1	2600	2611.40	191.2	2613	2619.05	230.1	2600	2605.50	220.7	2613	2615.55	201.4
ta04	15*15	2500	2500	2542.85	246.9	2501	2545.25	178.4	2500	2552.85	221.1	2501	2546.65	220.7	2500	2549.40	214.8
ta05	15*15	2600	2600	2657.80	281.6	2600	2658.05	232.2	2661	2670.60	212.3	2600	2647.10	245.0	2609	2662.70	220.2
ta06	15*15	2549	2549	2575.35	189.3	2584	2584.00	159.6	2549	2612.15	228.2	2584	2584.00	156.3	2549	2606.25	206.3
ta07	15*15	2684	2684	2723.05	254.7	2732	2747.65	249.4	2684	2721.25	278.9	2733	2747.35	223.3	2684	2719.10	223.7
ta08	15*15	2634	2634	2686.40	216.8	2634	2710.00	235.5	2654	2671.80	202.1	2634	2712.80	232.9	2634	2674.65	214.1
ta09	15*15	2637	2637	2638.65	184.0	2637	2637.00	162.7	2667	2694.05	202.3	2637	2637.00	170.8	2667	2696.60	209.5
ta10	15*15	2614	2629	2661.30	212.2	2667	2672.60	206.9	2614	2653.65	222.5	2629	2665.60	172.9	2629	2666.35	210.4
ta11	20*15	3041	3041	3146.15	647.1	3099	3182.65	594.8	3041	3161.40	688.7	3099	3186.45	600.8	3041	3169.15	666.9
ta12	20*15	3350	3350	3451.75	626.3	3350	3463.75	592.2	3418	3474.60	683.5	3350	3451.35	578.2	3394	3469.15	752.4
ta13	20*15	3028	3028	3145.05	596.4	3071	3102.75	647.7	3112	3214.45	757.9	3028	3100.45	573.2	3112	3209.35	605.5
ta14	20*15	3126	3126	3215.20	626.9	3133	3230.85	600.5	3140	3212.15	666.2	3126	3227.85	607.6	3165	3215.95	638.6
ta15	20*15	3064	3091	3140.50	663.7	3091	3142.20	599.3	3159	3242.30	785.0	3064	3135.80	564.7	3145	3245.75	638.4
ta16	20*15	3288	3374	3419.85	611.3	3288	3407.00	524.7	3310	3397.90	658.2	3288	3447.40	609.4	3374	3400.40	711.8
ta17	20*15	3247	3306	3440.90	738.1	3306	3418.40	643.6	3394	3467.05	682.1	3247	3423.10	548.2	3394	3456.45	684.1
ta18	20*15	3240	3240	3312.95	671.1	3277	3334.55	630.4	3240	3354.05	643.0	3268	3345.55	640.6	3240	3340.35	704.8
ta19	20*15	3099	3099	3224.05	626.5	3099	3246.50	694.5	3176	3213.40	671.7	3099	3267.45	647.4	3176	3245.50	690.1
ta20	20*15	3254	3254	3340.70	639.8	3295	3378.15	579.8	3254	3335.35	595.6	3295	3379.15	626.2	3254	3307.20	627.9
Avg			2627.5	2683.67	536.4	2637.4	2690.10	485.5	2644.7	2695.24	544.5	2632.0	2690.24	488.7	2644.2	2694.62	545.0
# of Best			20			12			10			16			11		

Table 9

Kruskal-Wallis Test on BRPD and ARPD (other test problems).

	chi-squared	df	p-Value
Test on BRPD	6.6835	4	0.1536
Test on ARPD	0.65194	4	0.9571

compared algorithms in this study, to account for the relative speeds of the different processors. Therefore, the computational times in Tables 4 and 5 are adjusted using the following formula.

$$\text{Original CPU time} \times (\text{score of old computer} / \text{score of new computer})$$

where the new computer is the one that was used in this study to execute the MSA-BST algorithm, yielding a score of 1165. The old computer was used by Sundar et al. (2017) to execute the CLLM, MCLM and HABC, yielding a score of 818.

Table 4 presents the best relative percentage deviation (BRPD), the average relative percentage deviation (ARPD) and the average CPU time in seconds (ACT) for each of the 21 small problem instances. As shown in Table 4, the MSA-BST found 21 BKSs; CLLM, MCLM and HABC obtained 20, 16 and 17 of the 21 BKSs, respectively. Table 4 shows that TEM_F and TEM_B can find the BKSs for 16 and 13 of the 21 test problems, respectively, supporting the claim that the proposed bi-directional shift timetabling procedure can decode better schedules than the classical forward timetabling procedure. The total average BRPD across the 21 small problem instances that was obtained using the proposed MSA-BST algorithm was 0.00%, whereas the corresponding values that were obtained using CLLM, MCLM, HABC, TEM_F and TEM_B were 0.01%, 0.41%, 0.23%, 0.23% and 0.38%, respectively. The total ARPD of the 21 small problem instances that was obtained using the proposed MSA-BST algorithm was 0.08%, whereas the corresponding values that were obtained using CLLM, MCLM and HABC

were 0.27%, 0.47% and 0.51%, respectively. These results reveal that the proposed MSA-BST algorithm outperforms the state-of-the-art algorithms in solving the small problem instances. It can be found that TEM_F and TEM_B are unable to obtain the best solution for some instances. That is, if the starting processing time of the first operation of a job is not delayed for one unit of time, the best solution cannot be obtained, even if the TEM_F and TEM_B are the total enumeration method for forward and backward timetabling.

Table 5 presents the BRPDs, ARPDs and ACTs that were obtained using the compared algorithms for the large problem instances. Since neither TEM_F nor TEM_B can obtain solutions within a reasonable computational time, they are not listed in Table 4. For the 40 large problem instances, the MSA-BST, CLLM, MCLM and HABC yielded 34 (34/40 = 85.0%), 8 (8/40 = 5.0%), 8 (8/40 = 5.0%) and 18 (18/40 = 45.0%) BKSs, respectively. Clearly, the proposed MSA-BST algorithm obtained the most BKSs among the four compared algorithms. Among the 34 BKSs that found by MSA-BST, there are 15 new BKSs, which are not found in the literature. The details of all solutions that obtained by the proposed MSA-BST algorithm have been put on a website (<http://swlin.cgu.edu.tw/data/nwjsp.zip>). The total average BRPD of the 40 large problem instances that was obtained using MSA-BST was 0.08%, whereas the corresponding values that were obtained using CLLM, MCLM and HABC were 2.74%, 1.33% and 0.81%, respectively. The total ARPD of the 40 large problem instances was 1.74% for the proposed MSA-BST algorithm, whereas for CLLM, MCLM and HABC, the corresponding values were 5.74%, 2.90% and 1.95%, respectively. Evidently, the proposed MSA-BST algorithm yielded better solutions than the other best available algorithms for large problem instances.

To confirm that the MSA-BST algorithm significantly outperformed the CLLM, MCLM, HABC, TEM_F and TEM_B algorithms, Kruskal-Wallis tests were performed in terms of BRPD and total ARPD. The analytical

results in Table 6 indicate that, at a confidence level of $\alpha = 0.05$, there are significant differences among 6 heuristics for the BRPD and total ARPD for small problem instances. The analytical results in Table 7 indicate that, at a confidence level of $\alpha = 0.05$, there are significant differences among the four compared algorithms for the BRPD and total ARPD with respect to larger problem instances.

To verify the effectiveness of the forward/backward and with/without 1 time-unit delay shift timetabling methods, another 29 instance problems are used. The instances abz5-abz9 are taken from Adams, Balas, and Zawack (1988), yn1-yn4 are taken from Yamada and Nakano (1992), and tai01-tai20 are taken from Taillard (1988). Table 8 shows the results obtained for these test problems. To further confirm the effectiveness of the forward/backward, with/without 1 time-unit delay shift timetabling methods, Kruskal-Wallis tests were applied to M_0 , M_1 , M_2 , M_3 and M_4 , where M_0 denotes the proposed MSA-BST algorithm, M_1 denotes the MSA algorithm with the forward non-delay timetabling procedure, M_2 denotes the MSA algorithm with the backward non-delay timetabling procedure, M_3 denotes the MSA algorithm with the forward delay timetabling procedure, and M_4 denotes the MSA algorithm with the backward delay timetabling procedure. The analytical results in Table 9 indicate that, at a confidence level of $\alpha = 0.05$, there is no significant difference among the forward/backward, with/without 1 time-unit delay shift timetabling methods. However, the proposed MSA-BST (M_0) obtained the highest number of best-known solutions, compared with its variants (i.e., M_1 , M_2 , M_3 and M_4).

5. Conclusions and recommendations for future studies

This study proposed an effective MSA-BST algorithm for solving the $J_m|nwt|C_{\max}$ problem, which is a challenging task that has received little attention. The proposed MSA-BST algorithm utilizes a bi-directional shift timetabling procedure to enlarge the search space and thus find better solutions. The computational results that were obtained using a set of benchmark instances verify the superiority of the MSA-BST algorithm over the best existing meta-heuristic in the literature. The proposed MSA-BST algorithm yields BKSs for all small problem instances and 34 out of 40 large problem instances. Also, MSA-BST yields new BKSs for 15 of the instances that are not found in the literature. The above experiments indicate that the proposed MSA-BST algorithm is a new and effective alternative algorithm for solving the $J_m|nwt|C_{\max}$ problem.

A few interesting relevant topics concerning NWJSPs remain open. First, this work could be extended by trying to find an improved timetabling procedure for the $J_m|nwt|C_{\max}$ problem. Second, developing other meta-heuristics for solving the $J_m|nwt|C_{\max}$ problem would be valuable. Third, more NWJSPs with alternative objective functions or multi-objective functions must be addressed. Fourth, more research on NWJSPs with other performance metrics is required. Finally, complex problems like stochastic and dynamic NWJSPs warrant investigation.

CRedit authorship contribution statement

Kuo-Ching Ying: Conceptualization, Writing - review & editing, Writing - original draft. **Shih-Wei Lin:** Supervision, Software, Validation.

Acknowledgment

The work of S.-W. Lin was supported in part by the Ministry of Science and Technology, Taiwan, under Grant MOST107-2410-H-182-005-MY2, and in part by the Linkou Chang Gung Memorial Hospital under Grant BMRPA19. The work of the K-C Ying is partially supported by the Ministry of Science and Technology of the Republic of China, Taiwan, under Grant MOST108-2221-E-027-025.

References

- Adams, J., Balas, E., & Zawack, D. (1988). The shifting bottleneck procedure for job shop scheduling. *Management Science*, 34, 391–401.
- AitZai, A., Benmedjdoub, B., & Boudhar, M. (2016). Branch-and-bound and PSO algorithms for no-wait job shop scheduling. *Journal of Intelligent Manufacturing*, 27, 679–688.
- Allahverdi, A. (2016). A survey of scheduling problems with no-wait in process. *European Journal of Operational Research*, 255, 6658–6686.
- Bansal, N., Mahdian, M., & Sviridenko, M. (2005). Minimizing makespan in no-wait job shops. *Mathematics of Operations Research*, 30, 817–831.
- Beasley, J. E. (1990). Or-library: Distribution test problems by electronic mail. *Journal of the Operational Research Society*, 41, 1069–1072.
- Bożejko, W., & Makuchowski, M. (2009). A fast hybrid tabu search algorithm for the no-wait job shop problem. *Computer and Industrial Engineering*, 56, 1502–1509.
- Bożejko, W., & Makuchowski, M. (2011). Solving the no-wait job-shop problem by using genetic algorithm with automatic adjustment. *International Journal Advanced Manufacturing Technology*, 57, 735–752.
- Bożejko, W., & Wodecki, M. (2006). New concepts in neighborhood search for permutation optimization problems. In E. K. Burke, & H. Rudová (Eds.). *PATAT 2006* (pp. 363–366). ISBN 80-210-3726-1.
- Brizuela, C. A., Zhao, Y., & Sannomiya, N. (2001). No-wait and blocking job-shops: Challenging problems for GA's. *IEEE international conference on systems, man, and cybernetics* (pp. 2349–2354). New York: IEEE.
- Chen, J. F. (2006). Approaches for the vehicle routing problem with simultaneous deliveries and pickups. *Journal of the Chinese Institute of Industrial Engineers*, 23, 141–150.
- Framinan, J. M., & Schuster, C. J. (2006). An enhanced timetabling procedure for the no-wait job shop problem: A complete local search approach. *Computers and Operations Research*, 33, 1200–1213.
- Fisher, R. A., & Yates, F. (1948). *Statistical tables for biological, agricultural and medical research* (3rd ed.). London: Oliver & Boyd 26–27.
- Hall, N., & Sriskandarajah, C. (1996). A survey on machine scheduling problems with blocking and no-wait in process. *Operations Research*, 44, 510–525.
- Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1993). Optimization by simulated annealing. *Science*, 220, 671–680.
- Li, X., Xu, H., & Li, M. (2016). A memory-based complete local search method with variable neighborhood structures for no-wait job shops. *International Journal Advanced Manufacturing Technology*, 87, 1401–1408.
- Lin, S. W., & Ying, K. C. (2015). A multi-point simulated annealing heuristic for solving multiple-objective unrelated parallel machine scheduling problem. *International Journal of Production Research*, 53, 1065–1076.
- Mascis, A., & Pacciarelli, D. (2002). Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research*, 143, 498–517.
- Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equations of state calculations by fast computing machines. *Journal of Chemical Physics*, 21, 1087–1092.
- Ozolins, A. (2018). A new exact algorithm for no-wait job shop problem to minimize makespan. *Operational Research-An International Journal*. <https://doi.org/10.1007/s12351-018-0414-1>.
- Pan, J. C. H., & Huang, H. C. (2009). A hybrid genetic algorithm for no-wait job shop scheduling problems. *Expert Systems with Applications*, 36, 5800–5806.
- Santosa, B., Budiman, M. F., & Wiratno, S. E. (2011). A cross entropy-genetic algorithm for m-machines no-wait job-shop scheduling problem. *Journal of Intelligent Learning Systems and Applications*, 3, 171–180.
- Schuster, C. J. (2006). No-wait job shop scheduling: Tabu search and complexity of subproblems. *Mathematical Methods of Operations Research*, 63, 473–491.
- Schuster, C. J., & Framinan, J. (2003). Approximate procedures for no-wait job shop scheduling. *Operations Research Letters*, 31, 308–318.
- Sundar, S., Suganthan, P. N., Jin, C. T., Xiang, C. T., & Soon, C. C. (2017). A hybrid artificial bee colony algorithm for the job-shop scheduling problem with no-wait constraint. *Soft Computing*, 21, 1193–1202.
- Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64, 278–285.
- Valenzuela, V. M., Brizuela, C. A., Cosío-León, M. A., & Romero-Ocaño, A. D. (2019). A combination of two simple decoding strategies for the no-wait job shop scheduling problem. In M. López-Ibáñez (Ed.). *Proceedings of the genetic and evolutionary computation conference (GECCO '19)* (pp. 864–871). New York, NY, USA: ACM.
- Vermeulen, H., Hoogeveen, H., & van den Akker, M. (2011). Solving the no-wait job shop problem: An ILP and CP approach. *Proceedings of Belgian/Netherlands artificial intelligence conference* (pp. 1–8).
- Yamada, T., & Nakano, R. (1992). A genetic algorithm applicable to large-scale job-shop instances. In R. Manner, & B. Manderick (Eds.). *Parallel instance solving from nature 2* (pp. 281–290). North-Holland.
- Zhu, J., & Li, X. (2012). An effective meta-heuristic for no-wait job shops to minimize makespan. *IEEE Transactions on Automation Science and Engineering*, 9, 189–198.
- Zhu, J., Li, X., & Wang, Q. (2009). Complete local search with limited memory algorithm for no-wait job shops to minimize makespan. *European Journal of Operational Research*, 198, 378–386.